

Scalable Distributed Computing - 2026/27

Lecture 6 - Workload Characterization and Experimental Design

Patrizio Dazzi patrizio.dazzi@unipi.it

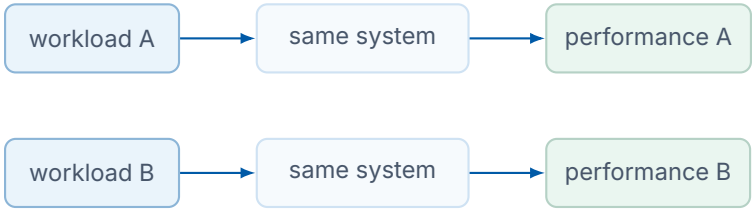
ACT I

From a performance number to a well-posed study

Before interpreting a number, identify the workload, the conditions, and the question.

Performance is not a property of a system in isolation

Consider the same distributed service under two different workloads:



The hardware and software may be unchanged, yet measured throughput and latency can be very different.

A performance number is meaningful only together with the workload under which it was obtained.

What do we mean by workload?

A **workload** describes the work presented to a system and the way that work arrives.

It may describe, for example,

- the requests, jobs, events, or operations to be processed;
- how much work arrives;
- when it arrives;
- what kinds of operations are mixed together;
- what data the operations act upon.

A workload may come from

production | a recorded trace | a synthetic generator.

Modelling note. A workload is not necessarily something we control. In production, we may simply observe it.

Performance is the observed response to a workload

For a given system configuration,

$$\text{workload} + \text{system configuration} \longrightarrow \text{observed performance}$$

Examples of performance observations include

throughput, latency, completion time.

Therefore a statement such as

“the system delivers 20,000 requests/s”

is incomplete unless we also know the conditions under which that value was measured.

Performance measurements describe a relationship, not an isolated property of the machine.

Observation and experiment answer different questions

Observation

We observe a running system:

$$W(t) \longrightarrow \text{system} \longrightarrow P(t).$$

We measure how workload and performance evolve, without deliberately changing the conditions.

Controlled experiment

We deliberately choose or vary selected conditions and measure the resulting performance.
For example,

$$N = 4, 8, 16, 32.$$

An experiment introduces deliberate control. A measurement campaign does not necessarily do so.

What is a performance experiment?

A performance experiment asks how an observed quantity changes when we deliberately change some condition.

A simple experiment has three parts:

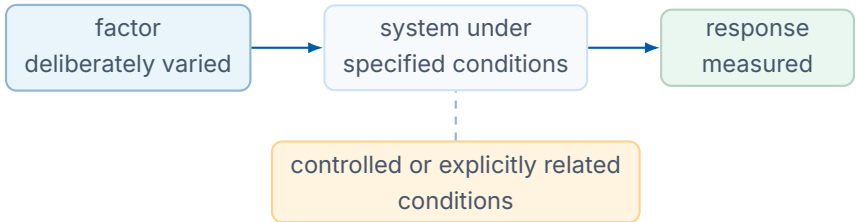
change | controlled conditions | measurement.

For example:

Change	Keep specified	Measure
number of nodes	workload definition	throughput
request load	system configuration	latency
data volume	request mix	completion time

The purpose of control is to make the result interpretable.

Three roles make an experiment interpretable



For example, in a load sweep:

r_{in} changes | configuration and request mix specified | X, p_{95} measured.

The experiment is interpretable when the role of each quantity is explicit.

A measurement can be correct and still answer an unclear question

Suppose we compare two runs:

	Run A	Run B
nodes	4	8
clients	20	80
dataset	100 GB	400 GB
throughput	12,000	21,000

The measurements may be perfectly accurate.
But what produced the difference?

more nodes? more offered work? more data?

Accurate measurements do not automatically produce an interpretable experiment.

Every performance study begins by defining the question

Before collecting measurements, specify

What changes?

What conditions are controlled or related to that change?

What response do we measure?

Examples of well-posed questions are:

- How does throughput change as cluster size increases?
- How does latency change as request load increases?
- How does completion time change as the degree of parallelism increases?

The experimental question determines which workload dimensions and performance metrics matter.

ACT II

Describing a workload

A request rate alone is not enough to characterize the work presented to a system.

A workload is more than ``how many requests''

Two workloads can contain the same number of requests and still exercise the system very differently.

A useful workload description must tell us something about

amount | timing | concurrency | operation mix | data

There is no single scalar quantity called “the workload”.

A workload specification is a vector, not a scalar

The dimensions introduced in this lecture can be collected as a compact description:

$$W = \left(r_T,\ \text{temporal structure},\ \bar{N}_T,\ \mathbf{p},\ D,\ \mathbf{s}, \ldots\right).$$

intensity r_T	timing / burstiness	concurrency $\bar{N}_T$	operation mix $\mathbf{p}$	data D	access shares $\mathbf{s}$

The notation is bookkeeping, not a claim that every system needs exactly these dimensions.

Modelling note. Keep the dimensions that can materially change the work presented to the system.

How to read a workload specification

A workload description collects several independent questions about the work presented to the system:

$$W = \left(r_T,\ \text{temporal structure},\ \bar{N}_T,\ \mathbf{p},\ \mathcal{D},\ \mathbf{s}, \ldots\right).$$

Component	What it describes	Typical interpretation
r_T	amount of arriving work	average arrivals per unit time over a window T
temporal structure	when that work arrives	steady, variable, periodic, bursty, impulsive
$\bar{N}_T$	simultaneous work	average number of requests concurrently in flight
$\mathbf{p}$	operation composition	fractions of reads, writes, queries, updates, ...
$\mathcal{D}$	data characteristics	dataset, working set, object sizes, data volume, ...
$\mathbf{s}$	access distribution	fraction of accesses directed to each item or partition

Two workloads may agree on one component and differ substantially on the others.

First dimension: how much work arrives?

Suppose a service receives

$$60{,}000 \text{ requests}.$$

The total amount of work is useful, but it does not tell us how quickly that work arrived.

Over an interval Δt, the observed average arrival rate is

$$\bar{r} = \frac{\text{requests arriving during } \Delta t}{\Delta t}.$$

For the example,

$$\bar{r} = \frac{60{,}000}{60} = 1{,}000 \text{ requests/s}.$$

Work volume and average arrival rate describe different properties of the workload.

Arrival events are discrete

In an observed trace, requests arrive at event times

$$t_1,\ t_2,\ \ldots,\ t_n,\ \ldots$$



At this level, each request is an event at a particular time. No continuous approximation has been made.

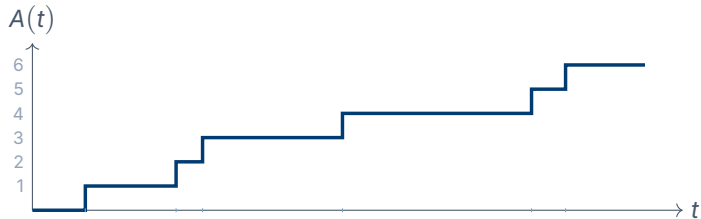
The raw temporal structure of a workload is a sequence of discrete arrival events.

A counting process accumulates the arrival events

Define

$$A(t) = \#\{i : t_i \leq t\},$$

the number of requests that have arrived up to time t.



For any window $[t, t+T]$,

$$\boxed{A(t+T) - A(t)}$$

is the exact number of observed arrivals in that window.

Exact discrete representation: no continuous approximation has been introduced yet.

The same events can be represented as impulses

A standard signal representation of the same arrivals is

$$\boxed{a(t) = \sum_i \delta(t - t_i)}$$

where δ is the **Dirac impulse**.

Each impulse has unit area, so integrating over a time interval counts the events inside it:

$$\int_u^v a(\tau)\, d\tau = \#\{i : u < t_i \leq v\}.$$

In the generalized-function sense,

$$\frac{dA(t)}{dt} = a(t).$$

Counting process and impulse train are two standard representations of the same discrete arrivals.

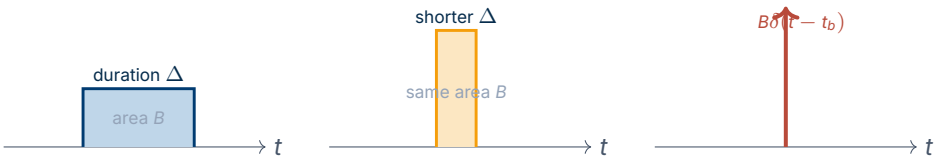
A Dirac impulse idealizes a short event with finite magnitude

Consider a burst of B requests spread uniformly over a short duration Δ:

$$b_\Delta(t) = \begin{cases} B/\Delta, & t_b \le t < t_b + \Delta, \\ 0, & \text{otherwise.} \end{cases}$$

Its total magnitude is independent of its duration:

$$\int_{-\infty}^{+\infty} b_\Delta(t)\, dt = B.$$



As $\Delta \to 0$ while the area remains B, the idealized limit is $B\delta(t - t_b)$.

From discrete arrivals to a fluid approximation

The exact event representation is

$$a(t) = \sum_i \delta(t - t_i).$$

When individual event times are less important than aggregate behavior at the chosen timescale, we replace the impulse train by a continuous non-negative rate function:

$$\boxed{\sum_i \delta(t - t_i) \quad \rightsquigarrow \quad r(t)}$$

with $r(t)$ measured in requests/s.

This is a **model transformation**: the real requests remain discrete.

A fluid model is a deliberate continuous relaxation of a discrete arrival process.

What does the fluid approximation preserve---and discard?

The approximation is useful only relative to the question and the chosen timescale.

Preserve

aggregate volume
temporal trend
rate variation at the modelling scale

Abstract away

exact event times
event-level discreteness
sub-timescale structure

A good approximation is not the one that preserves everything. It is the one that preserves what matters for the analytical question.

Continuous relaxation trades event-level detail for analytical tractability.

Integration over time accumulates the represented quantity

The calculus operation is always the same: integrate a time-dependent quantity to accumulate it.

integrand	units	time integral
arrival rate $r(t)$	requests/s	requests
impulse $B\delta(t - t_b)$	requests/s	B requests
concurrency $N(t)$	active requests	request-seconds

For example,

$$\underbrace{\int_t^{t+T} r(\tau)\, d\tau}_{\text{requests}} \quad \text{because} \quad \frac{\text{requests}}{\text{s}} \times \text{s} = \text{requests}.$$

The operator is shared; the physical meaning comes from the integrand and its units.

Average arrival rate exists in both representations

For the exact discrete trace,

$$\boxed{\bar{r}_T(t) = \frac{A(t+T) - A(t)}{T}}.$$

For the fluid approximation, define

$$\widetilde{A}(t) = \int_0^t r(\tau)\, d\tau,$$

so that

$$\boxed{\bar{r}_T(t) = \frac{\widetilde{A}(t+T) - \widetilde{A}(t)}{T} = \frac{1}{T}\int_t^{t+T} r(\tau)\, d\tau}.$$

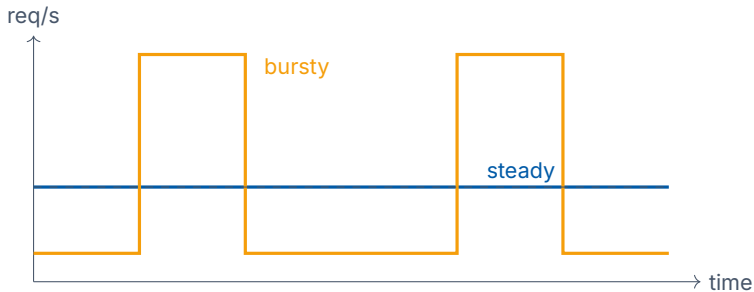
The first expression counts observed events; the second averages a continuous model.

Keep discrete observation and continuous modelling distinct, then compare them at the same timescale.

The same average rate can hide very different workloads

Both fluid workload models below deliver $60{,}000$ requests in one minute:

$$\bar{r} = 1{,}000 \text{ requests/s.}$$



The average arrival rate preserves volume over the window, not the temporal shape of the workload.

Worked example: equal volume, different temporal shape

For the steady workload,

$$\int_0^{60} r_{\text{steady}}(t)\,dt = 60 \cdot 1{,}000 = 60{,}000 \text{ requests.}$$

For the bursty workload,

$$\int_0^{60} r_{\text{bursty}}(t)\,dt = 40 \cdot 300 + 20 \cdot 2{,}400 = 60{,}000 \text{ requests.}$$

Hence both have

$$\bar{r}_{60} = \frac{60{,}000}{60} = 1{,}000 \text{ requests/s,}$$

although one reaches $2{,}400$ requests/s while the other never exceeds $1{,}000$.

Equal average arrival rate does not imply equal short-term pressure on the system.

Burstiness describes short-term concentration of offered work

A workload is **bursty** when work tends to arrive in concentrated periods rather than at a nearly uniform rate.

The statement

$$\text{average} = 1{,}000 \text{ req/s}$$

does not distinguish between

$$1{,}000,\ 1{,}000,\ 1{,}000,\ 1{,}000, \ldots$$

and

$$200,\ 200,\ 3{,}400,\ 200, \ldots$$

Modelling note. Burstiness is defined relative to a temporal scale; long-window aggregation can hide short-lived concentration.

An impulse is the limiting model of a burst

Suppose a background workload is described by a smooth rate $r_0(t)$. A burst of B requests whose duration is negligible at the modelling timescale can be idealized as

$$\boxed{r_b(t) = r_0(t) + B\delta(t - t_b)}.$$

For any interval containing t_b,

$$\int r_b(t)\, dt = \int r_0(t)\, dt + B.$$

If the burst duration is *not* negligible, use a finite-width rate profile instead of a Dirac impulse.

The impulse abstraction preserves the burst magnitude while discarding its internal duration.

A standard burst-plus-rate model bounds arrivals over any interval

Sometimes we do not need to predict exactly *when* a burst occurs. Instead, we want an upper bound on how much work can arrive in any interval of length u.

$$A(t+u) - A(t) \leq b + \rho u, \qquad u \geq 0$$

ρ
long-term or sustained rate

b
burst allowance

This affine **arrival curve** is a standard model in deterministic network calculus.

A burst can be represented as a specific event, or bounded without specifying its exact occurrence time.

The observation timescale changes the variability we can see

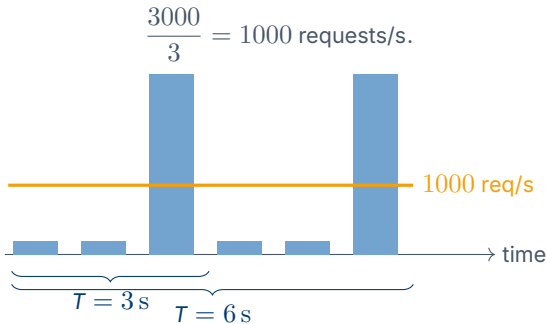
Consider arrivals counted in one-second bins:

$$200, \quad 200, \quad 2600, \quad 200, \quad 200, \quad 2600, \ldots$$

Each three-second interval contains

$$200 + 200 + 2600 = 3000$$

requests, hence its average rate is



Arrival rate and concurrency describe different things

Arrival rate describes how quickly new work enters the system.

requests arriving per unit time

Concurrency describes how much work is simultaneously in progress.

requests currently active in the system

Consider two services receiving the same

$1{,}000$ requests/s.

If requests remain active for very different amounts of time, the number of simultaneous requests can also be very different.

Rate is about arrivals over time; concurrency is about overlapping work.

Concurrency is also a time-varying quantity

Let

$$N(t) = \text{number of requests active in the system at time } t.$$

Unlike $r(t)$, $N(t)$ is not a rate: its unit is simply *active requests*.

Integrating it over time therefore accumulates *request-time*:

$$\int_{t_0}^{t_0+T} N(t)\,dt \qquad [\text{request-seconds}].$$

Dividing that accumulated occupancy by the window length gives average concurrency:

$$\boxed{\overline{N}_T = \frac{1}{T}\int_{t_0}^{t_0+T} N(t)\,dt}.$$

The same integration rule is being reused; dimensional analysis tells us what has been accumulated.

Worked example: build concurrency from request lifetimes

Consider four requests active during the intervals

$$R_1 : [0, 4), \qquad R_2 : [1, 3), \qquad R_3 : [2, 5), \qquad R_4 : [4, 6).$$

At each time, concurrency is the number of intervals covering that instant:

$$N(t) = \begin{cases} 1, & 0 \le t < 1, \\ 2, & 1 \le t < 2, \\ 3, & 2 \le t < 3, \\ 2, & 3 \le t < 5, \\ 1, & 5 \le t < 6. \end{cases}$$

Concurrency is obtained by counting overlapping active requests at each instant.

Worked example: compute average concurrency

Over the six-second window,

$$\overline{N}_6 = \frac{1}{6}\int_0^6 N(t)\,dt.$$

Using the piecewise values from the previous slide,

$$\int_0^6 N(t)\,dt = 1\cdot 1 + 2\cdot 1 + 3\cdot 1 + 2\cdot 2 + 1\cdot 1 = 11 \text{ request-seconds.}$$

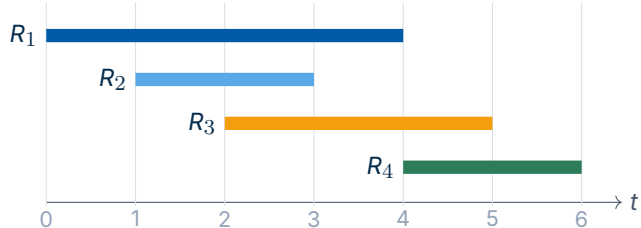
Therefore

$$\boxed{\overline{N}_6 = \frac{11}{6} \approx 1.83 \text{ active requests}}.$$

Average concurrency is a time-average of overlapping work, not an arrival rate.

Concurrency is the area created by overlapping request lifetimes

For the four-request example,



Each request contributes one unit of concurrency for the duration of its lifetime. Therefore,

$$\int_0^6 N(t)\,dt = \sum_{i=1}^{4} L_i = 4 + 2 + 3 + 2 = 11 \text{ request-seconds.}$$

The same quantity can be read vertically as concurrency or horizontally as accumulated request lifetimes.

The operation mix also defines the workload

Suppose two systems both receive

$$10{,}000 \text{ requests/s.}$$

But their workloads are:

	Workload A	Workload B
reads	95%	50%
writes	5%	50%

The arrival rate is identical.
The work requested from the system is not.

A workload description must preserve differences among operation types when those differences affect system work.

Operation mix can be represented quantitatively

Let p_k be the fraction of arrivals belonging to operation class k:

$$p_k \geq 0, \qquad \sum_{k=1}^{m} p_k = 1.$$

If the total arrival rate is r, then class k contributes

$$\boxed{r_k = p_k r}.$$

For $r = 10{,}000$ requests/s:

	Workload A	Workload B
r_{read}	$0.95r = 9{,}500$	$0.50r = 5{,}000$
r_{write}	$0.05r = 500$	$0.50r = 5{,}000$

Equal total arrival rate can hide very different class-specific arrival rates.

Workload also includes the data being accessed

Consider the same request stream against two datasets:

$$\boxed{10\ \text{GB}} \quad \text{versus} \quad \boxed{10\ \text{TB}}.$$

The request rate and operation mix may be identical, but the resulting execution conditions need not be.

Relevant workload properties can include

- dataset size;
- working-set size;
- object or record sizes;
- which data items are accessed.

The work is defined not only by the operations, but also by the data on which they operate.

Work may be distributed unevenly across the data

Suppose a service partitions its data across eight partitions.



A uniform workload might spread requests approximately evenly.
A **skewed workload** concentrates a disproportionate fraction of the work on a smaller subset of items or partitions.

The total load can remain unchanged while its distribution across the system changes.

Represent partition load as traffic shares

Let r_j be the arrival rate directed to partition j. Then

$$r = \sum_{j=0}^{7} r_j, \qquad \boxed{s_j = \frac{r_j}{r}}, \qquad \sum_{j=0}^{7} s_j = 1.$$

For eight partitions and total rate

$$r = 8{,}000 \text{ requests/s},$$

a uniform workload has

$$r_j = 1{,}000 \text{ requests/s} \implies \boxed{s_j = 0.125 = 12.5\%}$$

for every partition.

Traffic shares separate total workload intensity from how that workload is distributed.

Worked example: redistribute load without changing its total

Start from the same total arrival rate,

$$r = 8{,}000 \text{ requests/s.}$$

Instead of $1{,}000$ requests/s per partition, let

$$r_3 = 4{,}500, \qquad r_j = 500 \quad (j \neq 3).$$

The total is still

$$r_3 + \sum_{j \neq 3} r_j = 4500 + 7(500) = 8000.$$

Skew changes the distribution of offered work without requiring a change in total offered work.

Worked example: compute the skewed traffic shares

For the hot partition,

$$\boxed{s_3 = \frac{4500}{8000} = 0.5625 = 56.25\%}.$$

For every other partition,

$$\boxed{s_j = \frac{500}{8000} = 0.0625 = 6.25\% \qquad (j \neq 3)}.$$

Check the normalization:

$$0.5625 + 7(0.0625) = 1.$$

One partition receives more than half of all requests even though the total arrival rate is unchanged.

Worked example: compare skewed and uniform partition load

Under the uniform workload, every partition had share

$$s_j^{\text{uniform}} = 12.5\%.$$

Under the skewed workload,

$$s_3^{\text{skewed}} = 56.25\%.$$

Therefore partition P_3 now receives

$$\boxed{\frac{s_3^{\text{skewed}}}{s_3^{\text{uniform}}} = \frac{56.25}{12.5} = 4.5}$$

times its uniform-share load.

A distributional workload change can multiply the load seen by one partition without increasing total load.

A simple concentration statistic makes the skew visible

For partition shares $\mathbf{s} = (s_0, \ldots, s_7)$, define

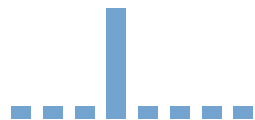
$$s_{\max} = \max_j s_j.$$

Uniform



$s_{\max} = 0.125.$

Skewed



$s_{\max} = 0.5625.$

Modelling note. $s_{\max}$ is only one descriptor of concentration; it does not replace the full share vector $\mathbf{s}$.

A workload is a combination of dimensions

Two workloads with the same average arrival rate may still differ in

temporal structure | concurrency | operation mix

data volume | access skew.

Therefore,

$$\text{same average rate} \not\Rightarrow \text{same workload.}$$

To interpret performance, we need enough workload dimensions to describe the work that actually reaches the system.

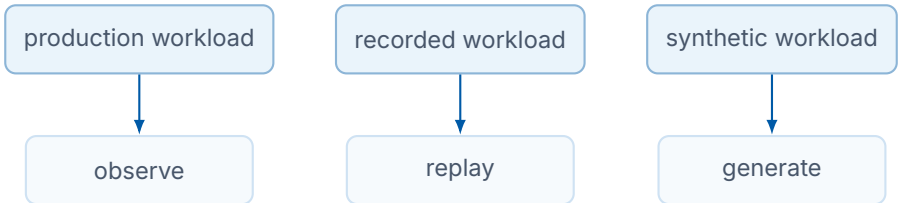
ACT III

Observing and generating workload

A workload may come from users, traces, or a workload generator.

Where does the workload come from?

There are three common situations:



These approaches give us different degrees of control over the workload.

Measurement does not require a synthetic benchmark.

A workload generator can interact with the system

Consider one client repeatedly doing this:

send request $\rightarrow$ wait for reply $\rightarrow$ send next request.

If the system becomes slower, the client waits longer.
Therefore it also sends new requests less frequently.

slower system $\Longrightarrow$ lower generated arrival rate.

The system can influence the workload produced by the generator.

Closed-loop workload generation

The previous client is a simple **closed-loop generator**.

request → system → response → next request.

With K such clients, we can control the number of clients,

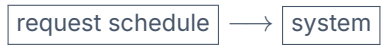
$$K,$$

but the resulting arrival rate also depends on how quickly the system completes requests.

In a closed loop, concurrency may be controlled more directly than arrival rate.

Open-loop workload generation

An **open-loop generator** schedules new requests independently of when previous requests complete.



For example, the generator may attempt to send

$$r = 1{,}000 \text{ requests/s}$$

whether the system is responding quickly or slowly.

In an open loop, the offered arrival rate is controlled independently of the system's response time.

Open and closed loops can expose different behavior

Suppose the system becomes slower.

Closed loop

slower replies



fewer new requests

The offered workload can decrease.

Open loop

slower replies



scheduled arrivals continue

The offered workload need not decrease.

The workload-generation method is part of the experimental setup.

Worked example: a closed-loop client can hide a slowdown

Consider one synchronous client with negligible think time:

send $\rightarrow$ wait $\rightarrow$ receive $\rightarrow$ send next.

Fast replies

$L = 50$ ms

$$r_{\text{client}} \approx \frac{1}{0.050} = 20 \text{ requests/s.}$$

Slow replies

$L = 200$ ms

$$r_{\text{client}} \approx \frac{1}{0.200} = 5 \text{ requests/s.}$$



A closed-loop generator can reduce offered load precisely when the system becomes slower.

Offered load and completed throughput are different quantities

A generator may offer work at rate

$$r_{\text{in}},$$

while the system completes work at rate

$$X.$$

These quantities answer different questions:

r_{in} : work presented to the system

X : work completed by the system

Do not use “requests per second” without saying whether it refers to arrivals or completions.

A reproducible workload description answers eight questions

unit of work
what?

intensity
how much?

timing / scale
when?

concurrency
how much overlap?

operation mix
which classes?

data
which and how much?

generation
observe, replay, generate?

interaction
open or closed?

Not every study needs every field at the same level of detail. The criterion is simpler:

Specify enough workload structure that another analyst can understand what work actually reached the system.

We can now separate workload from system response

So far we have described the **input**:

rate, timing, concurrency, mix, data, skew.

We have also seen that the way workload is generated matters.

The next question is:

How do we describe the system's observed response?

Workload characterization comes before performance interpretation.

What we can now do

We can now:

- distinguish an observed event trace, its counting process, and an impulsive representation;
- introduce a fluid approximation explicitly when aggregate behavior matters more than individual events;
- use integration and dimensional analysis consistently to move from rates or states to accumulated quantities;
- reason about temporal concentration using finite bursts, impulsive bursts, and burst-plus-rate bounds;
- preserve operation mix, data volume, skew, concurrency, and workload-generation semantics;
- specify offered work precisely enough for a reproducible performance study.

The next lecture starts from the other side of the boundary: how do we measure the system's response?

Selected references for further study

- D. J. Daley and D. Vere-Jones, *An Introduction to the Theory of Point Processes*, 2nd ed., Springer.
- P. Brémaud, *Point Process Calculus in Time and Space: An Introduction with Applications*, Springer, 2020.
- W. Whitt, *Stochastic-Process Limits: An Introduction to Stochastic-Process Limits and Their Application to Queues*, Springer, 2002.
- J.-Y. Le Boudec and P. Thiran, *Network Calculus: A Theory of Deterministic Queuing Systems for the Internet*, Springer LNCS 2050, 2001.
- D. J. Lilja, *Measuring Computer Performance: A Practitioner's Guide*, Cambridge University Press, 2000.

These tools come from established point-process, fluid-model, network-calculus, and performance-analysis traditions.



Questions or comments?

L6 - Workload Characterization and Experimental Design

Department of Computer Science - University of Pisa